AWS SAMPLE · AMAZON BEDROCK AGENTCORE

Agentic phishing URL detection

Paste a link. Get an explained verdict, the evidence behind it and a plain-language report, in your own AWS account.

Amazon Bedrock AgentCore | Claude Opus 5.5 + Sonnet 5.5 | Strands Agents | AWS CDK (Python) | MIT-0

THE PROBLEM

Every reported link needs someone to open it safely and decide.

RISKY TO OPEN

The page itself is the threat

Checking a lure means loading it somewhere it cannot reach your machine or your network.

MANY SIGNALS

The link, the page and the code

Look-alike domains, cloned sign-in pages and forms that post elsewhere each need a different eye.

NEEDS A REASON

A verdict alone is not enough

An analyst or a publishing gate needs the evidence, so a person can review the call.

WHAT IT DOES

One link in. A verdict you can check.

- Phishing · block
- Suspicious · warn
- Legitimate · allow
- Error 404 · Site down · Unknown

SCAN

One URL in the web console

Verdict, risk score, screenshot, redirect chain, code insights and a report.

BATCH

Paste a list or upload a CSV

Up to 200 URLs in the app, up to 1,000 through the API, with live progress.

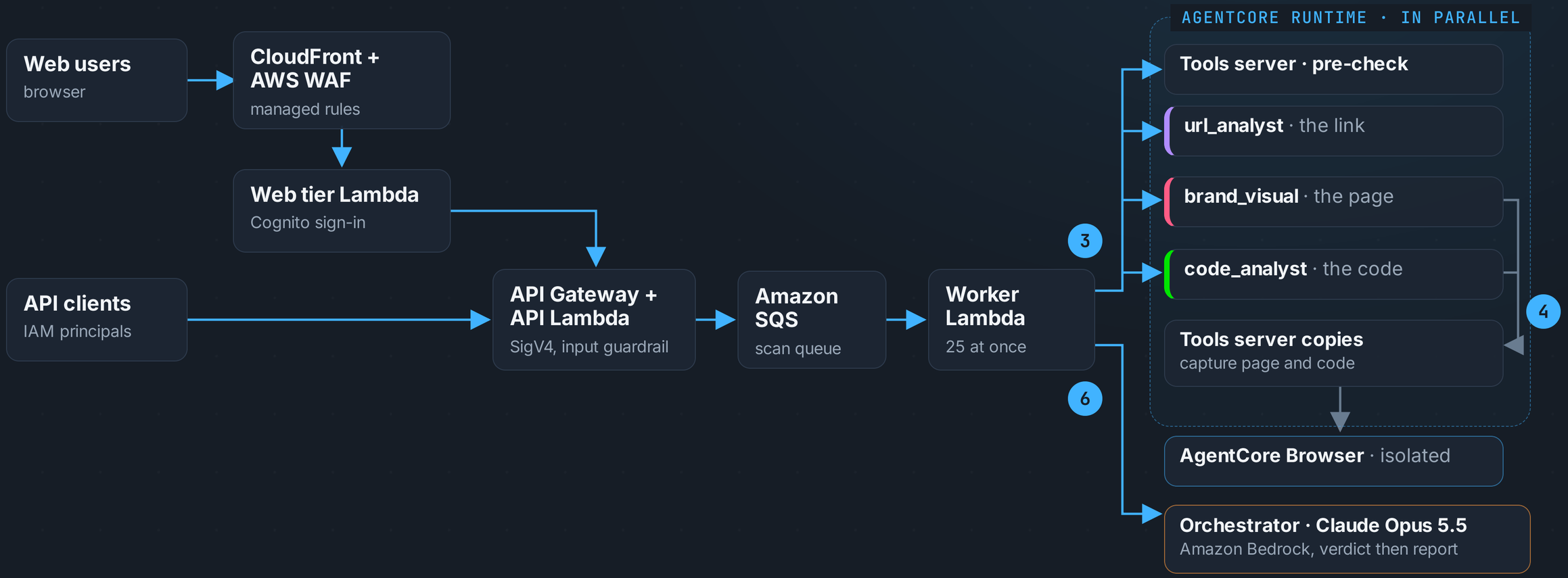
API

SigV4-signed REST API

The same API the console uses, ready for a pre-publish gate.

ARCHITECTURE · DEFAULT PATH

The page only ever opens in AgentCore Browser



SUPPORTING | DynamoDB | Amazon S3 | AgentCore Memory | Bedrock Guardrails | AWS KMS | CloudWatch | Harness path: opt-in

Source: infra/ and docs/architecture.md

HOW A SINGLE SCAN FLOWS

Six steps from paste to verdict

1

Submit

In the console or the API. A guardrail checks the URL, then it is stored and queued.

2

Queue

The worker takes it from Amazon SQS, up to 25 scans at once by default.

3

Pre-check + 3 specialists

All start together. A 404 or a dead site ends here with a fixed answer.

4

Capture

The page specialists capture the page through their own tools server copy.

5

Isolated browser

AgentCore Browser loads it. Nothing is typed, submitted or solved.

6

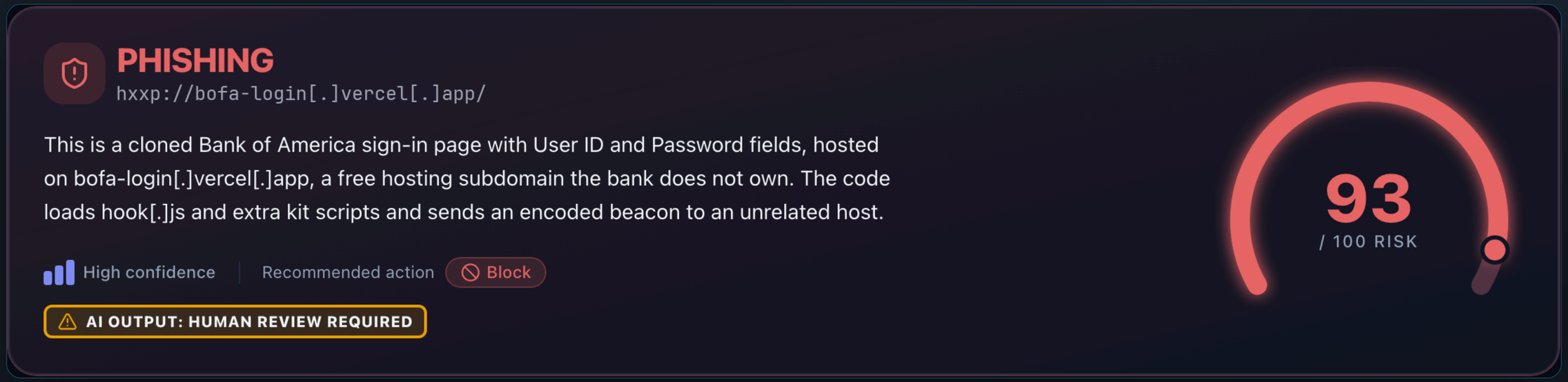
Judge, then explain

The orchestrator returns the verdict first, then a report a guardrail screens.

Source: README.md, docs/architecture.md

THE VERDICT

Verdict first. Evidence and report right behind it.



Risk score 0 to 100 **Action** block, warn or allow **Report** written after the verdict, screened

Web console, dark theme. URL defanged for this slide.

THREE SPECIALISTS, ONE JUDGE

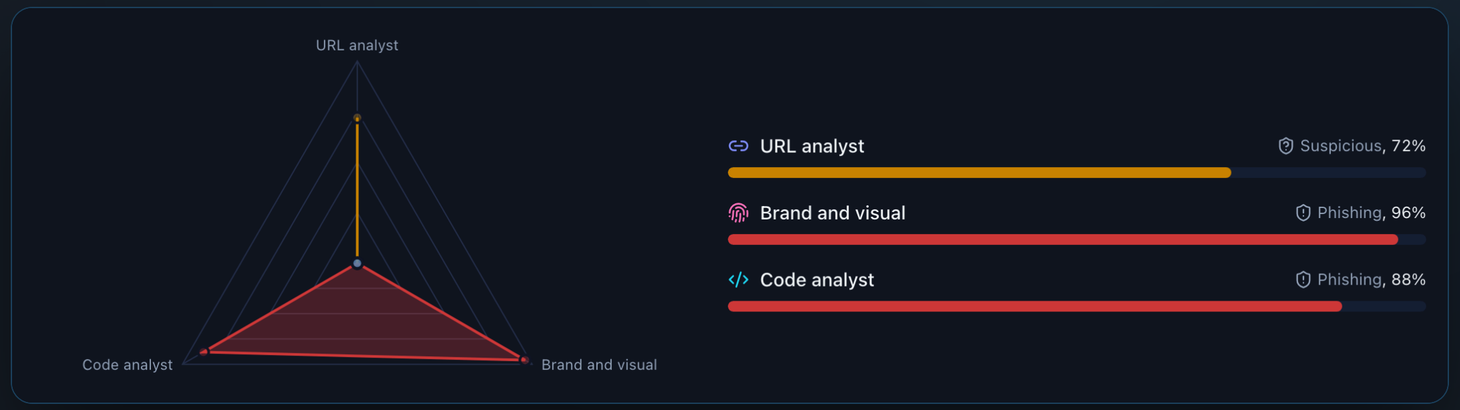
Each agent checks one thing

url_analyst
Look-alike domains, risky hosting, domain age, redirects

brand_visual
Brand impersonation, fake sign-in pages, scare copy

code_analyst
Forms posting elsewhere, keyloggers, obfuscation

orchestrator
Judges the findings, gives the verdict, writes the report



In this scan the specialists disagreed: the link alone looked suspicious, the page and its code looked like phishing. The orchestrator weighed the evidence and returned **phishing, risk 93**.

Specialists: Claude Sonnet 5.5 · Orchestrator: Claude Opus 5.5

BATCH

Hundreds of links, one queue, one summary

01 · IN

Paste URLs or upload a CSV. Up to 200 in the app, 1,000 by API.

02 · QUEUE

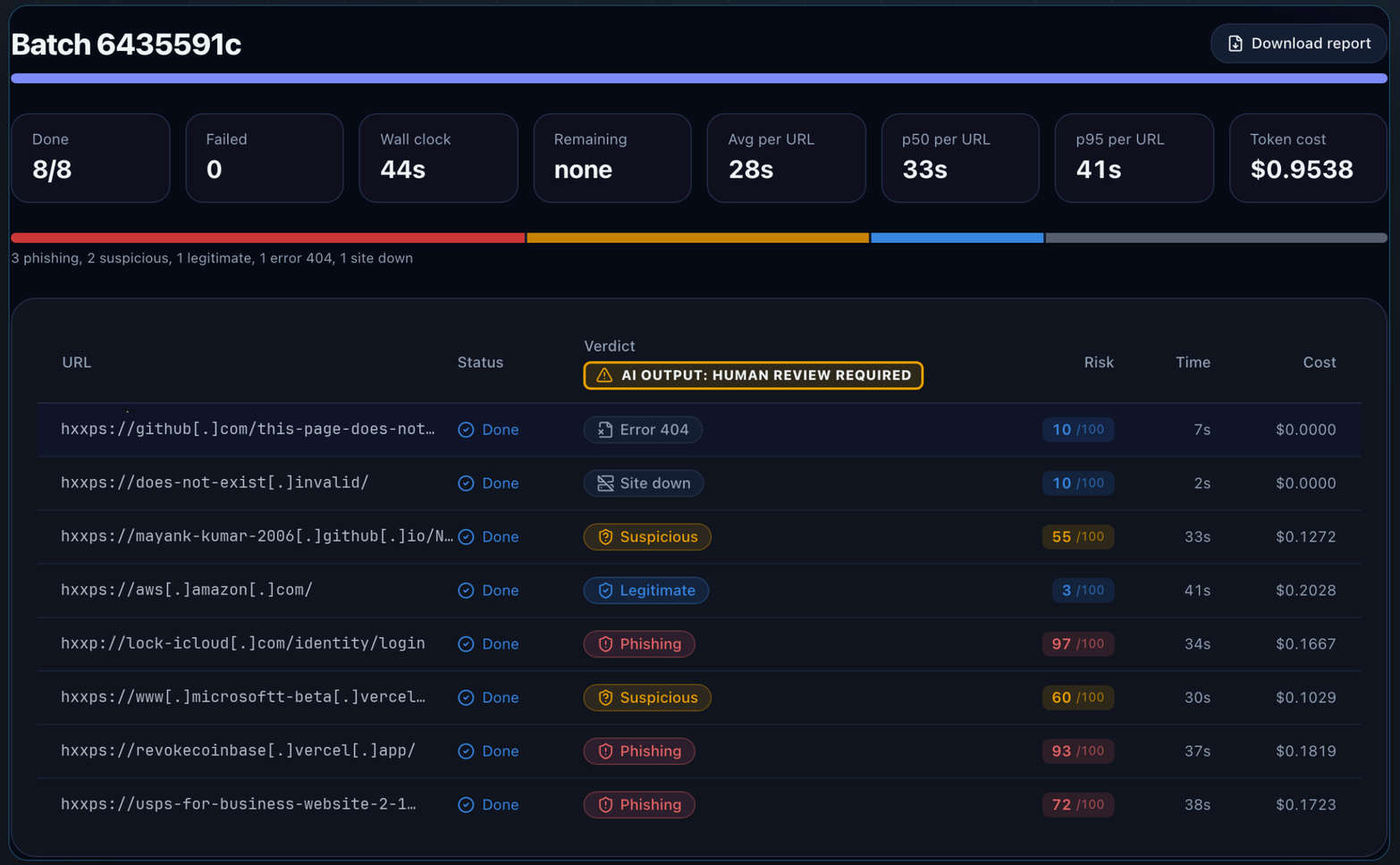
One scan per URL on the same SQS queue and worker.

03 · PROGRESS

Counts by verdict, timing and cost update every 2 seconds.

04 · SUMMARY

A per-URL table and a downloadable report.



Web console, earlier test run. URLs defanged for this slide.

API

Every route is SigV4. Post, poll, read.

README.md · Use the API

```
id=$(awscurl --service execute-api --region "$AWS_REGION" -X POST "$API/scan" \
  -H 'Content-Type: application/json' -d '{"url": "https://example.com"}' | jq -r .scan_id)

# poll until the scan is COMPLETE or FAILED
until awscurl --service execute-api --region "$AWS_REGION" "$API/scans/$id" > scan.json &&
  jq -e '.status == "FAILED" or .status == "COMPLETE"' scan.json > /dev/null; do sleep 3; done

jq '{status, verdict: .verdict.verdict, risk: .verdict.risk_score,
     action: .verdict.action, page_observed: .verdict.page_observed}' scan.json
```

Pre-publish gate: hold for a person when the scan failed, the action is warn or block, the risk is 70 or more, or the page was not observed.

Batches, idempotent retries and lookups: docs/api.md

MEASURED RESULTS · LATENCY FLAGS ON VS OFF

About twice as fast and a third cheaper, same accuracy

MEDIAN SCAN TIME

20 to 23 s

all on, two runs · vs **41.5 s** all off

MODEL COST PER SCAN

$0.09

mean, all on · vs **$0.13** all off

PHISHING CAUGHT

47 /52

on and off alike · false positives **2 of 50** in both

Of the latency flags, only `directOrchestration` has a clearly measured effect on its own: off, it added about 8 to 11 s at the median and about 47% more cost. The others were inside run-to-run noise and are not separately measured.

2026-11-03, test stack, us-west-2, 110 URLs (60 phishing, 50 legitimate; 8 dead left out), episodic memory off, 25 scans in flight. Scan time is worker start to verdict. Source: docs/evaluation.md.

MEASURED RESULTS · 407 LIVE URLS

What a pre-publish gate would hold

PHISHING CAUGHT (RECALL)

0.883

166 of 188 phishing URLs

FALSE POSITIVE RATE

0.046

10 of 219 legitimate URLs

SCAN TIME, MEDIAN / P90

18.6 s

p90 **31.2 s** · before: 37.8 s

Caught means a phishing verdict, or suspicious with risk 50 or more. The labelled set is private, so reproduce the method with your own labelled URLs.

2026-11-03, test stack, us-west-2, 448 URLs submitted at once (224 from a public phishing feed, 224 legitimate), 407 live, episodic memory off, 25 in flight. Source: README.md, docs/evaluation.md.

SECURITY BY DESIGN

Built to be careful with hostile pages

SIGNED ACCESS ONLY

The API takes SigV4 (IAM) calls only. The console signs in with Amazon Cognito, TOTP MFA optional or required, behind CloudFront and AWS WAF.

NEVER ON YOUR MACHINE

Pages open in AgentCore Browser. Agents never type, submit forms or solve bot checks. Every outbound request passes an SSRF guard.

GUARDRAILS BOTH WAYS

Amazon Bedrock Guardrails check every submitted URL and every report. URLs in model text are defanged.

ENCRYPTED AND SCOPED

AWS KMS customer managed keys on tables, buckets, queues and logs. One IAM role per component, each wildcard explained.

Details: docs/security.md, docs/threat-model.md

RESPONSIBLE USE

The verdict is advisory. A person makes the call.

LABELLED AS AI

Every result carries "AI-generated analysis: human review required".

HONEST ABOUT GAPS

If a warning or bot check hid the page, `page_observed` is false and the verdict stays at suspicious.

SAMPLE CODE

For learning and prototyping, not production ready. Review and test before real use.

COST

Pennies per scan, a small idle floor

IDLE STACKS

$14 to $15

per month, estimate · mostly AWS WAF and KMS

PER SCAN

$0.105

$0.090 tokens, measured + **about $0.015** other services, estimate

1,000 SCANS A MONTH

$120 to $130

computed: measured tokens plus estimates, with storage

Every resource is tagged with `Project` and `Owner`, so spend shows up in Cost Explorer. The Insights page shows token spend per URL, batch and model.

Assumptions: us-west-2 application and us-east-1 edge stack, on-demand prices, no free tier; token mean measured 2026-11-03 on 407 live URLs. Per-service breakdown: docs/cost.md.

DEPLOY AND CLEAN UP

Three commands up. One command down.

deploy · about 8 minutes the first time

```
# 1. get the code and dependencies
git clone https://github.com/aws-samples/sample-agentic-phishing-detection.git
cd sample-agentic-phishing-detection && uv sync --all-packages

# 2. once per account
npx aws-cdk@2.1143.0 bootstrap aws://<account-id>/$AWS_REGION \
  aws://<account-id>/us-east-1

# 3. deploy, tagged with your team
cd infra && npx aws-cdk@2.1143.0 deploy --all -c owner=<team>
```

clean up

```
cd infra
npx aws-cdk@2.1143.0 destroy --all
```

Deletes stored scans and every Cognito user. Then confirm nothing tagged `Project=phish-triage` is left in both Regions.

NEXT STEPS

Try it on your own links

01

Deploy in a sandbox

One CDK command, in your own account.

02

Measure on your data

Score it against links you have already labelled.

03

Pilot a gate

Hold risky links for review through the API.

04

Tune and clean up

Models, concurrency and retention are deploy options.

github.com/aws-samples/sample-agentic-phishing-detection

MIT-0 · sample code, not for production use